Meridian Global Airlines Case Study

Meridian Global Airlines — Predictive Maintenance & Irregular Operations Multi-Agent Platform

Confidential Internal Engagement Documentation

Engagement Period: March 2025 – February 2026

1. Executive Summary

Meridian Global Airlines (MGA), a 340-aircraft international carrier operating out of four hub airports, engaged an Enterprise AI Architecture team to design and deliver an agentic AI platform spanning two linked problem domains: **predictive maintenance** for the widebody and narrowbody fleets, and **irregular operations (IROPS) recovery** — the process of re-accommodating crew, aircraft, and passengers when weather, mechanical, or crew-legality events disrupt the published schedule.

The engagement ran eleven months, from initial discovery through production rollout across two hubs, followed by a phased expansion. It produced a multi-agent architecture built on event-driven telemetry ingestion, a maintenance-prediction agent, a crew-legality agent, a schedule-recovery orchestrator, and a human-in-the-loop dispatcher cockpit — all governed by a safety-case framework co-developed with MGA's flight operations and airworthiness teams.

The engagement is notable for a production incident four months post-launch in which the recovery orchestrator issued a crew reassignment that violated FAA duty-time regulations before a human caught it — a near-miss that reshaped MGA's human-in-the-loop policy and became the template other airlines in the holding company later adopted. This case study documents the full arc: discovery, architecture, the political fight over agent autonomy, the incident, and the governance model that emerged from it.

Key outcomes:

- 22% reduction in unscheduled maintenance events on monitored fleet segments over nine months post-rollout
- Average IROPS recovery plan generation time reduced from 47 minutes (manual dispatcher process) to 6 minutes for the assisted workflow
- One Class B safety-adjacent incident (near-miss, no regulatory violation reached passengers/crew) triggering a full HITL redesign
- Sustained agent-generated recovery plans in production with mandatory human sign-off retained permanently for any plan touching crew duty-time or MEL (Minimum Equipment List) dispatch decisions

---

2. Client Background

Meridian Global Airlines is a mid-size international carrier (fictionalized composite for this case study) with:

- 340 aircraft across three fleet types: A320 family, A350, and a legacy 737NG fleet being phased out
- Four hubs: two in North America, one in Europe, one in the Middle East
- An in-house MRO (maintenance, repair, and overhaul) division handling roughly 60% of line and base maintenance, with the remainder outsourced to two contracted MRO partners
- A 22,000-employee workforce including approximately 4,800 pilots and 7,200 flight attendants under two separate union contracts with materially different duty-time and reserve rules
- Legacy crew scheduling running on a 20-year-old mainframe-adjacent system (Jeppesen Crew Manager derivative) with a brittle integration layer
- A newly hired CTO (Priya Nadarajah, arrived 14 months prior) mandated by the board to modernize the technology estate after two embarrassing IROPS meltdowns during winter storms in the prior two years, each costing an estimated $30–40M in passenger compensation, hotel/crew accommodation, and lost revenue

MGA's Chief Architect, Tom Reyes, had spent the prior year building a case internally for an "AI-first operations" strategy but had been repeatedly blocked by the CISO (Elena Vasquez) over concerns about giving autonomous systems any decision authority over safety-adjacent operations, and by the VP of Flight Operations, Capt. Daniel Okoro, who was openly skeptical of "black box" recommendations in a domain governed by strict regulatory process.

---

3. Business Problem

Two connected pain points drove the engagement:

Predictive maintenance. MGA's maintenance program was largely reactive/scheduled rather than predictive. Unscheduled maintenance events (an aircraft going AOG — Aircraft On Ground — outside of planned maintenance windows) were costing an estimated $18M annually in direct disruption costs alone, before accounting for downstream schedule cascades. The airline had ACARS and engine OEM telemetry (Pratt & Whitney and CFM data feeds) but no unified analytics layer; each data source lived in a separate vendor portal, and mechanics were making dispatch decisions largely on gut instinct plus paper job cards.

IROPS recovery. When a disruption hit — a diverted flight, a sick crew member, a maintenance AOG — MGA's dispatchers manually reconstructed the recovery plan: which crews were legal to operate which flights, which aircraft could be swapped, which passengers needed rebooking, and in what sequence. During a major weather event, MGA might run 40+ simultaneous disruptions, and the manual process routinely broke down, producing crew-legality violations that had to be caught after the fact, cascading misconnects, and enormous compensation exposure.

Both problems shared a root cause the discovery phase surfaced explicitly: **decision-relevant data existed but was fragmented across systems that didn't talk to each other in real time**, and the humans making decisions were reasoning under time pressure without machine assistance.

---

4. Constraints

The following constraints were established in the first month and revisited repeatedly throughout the engagement — several of them materially reshaped the architecture:

1. **Regulatory.** Any system touching crew duty-time legality had to be auditable against FAA Part 117 (and EASA FTL for the European hub) with a defensible decision trail. The airline's legal counsel was explicit: an AI system could *recommend* a crew assignment but a certificated dispatcher or crew scheduler had to be the accountable decision-maker of record for anything filed with regulators.
2. **Safety case.** Flight Operations required a formal safety case (modeled on ARP4754A-style system safety argumentation, adapted for software) before any AI output could influence dispatch decisions, even indirectly.
3. **Union agreements.** Crew rules were not just regulatory minimums but negotiated contract terms (e.g., minimum rest, trip-pairing fairness, reserve call-out order) that varied by union and had to be encoded precisely — errors here created grievances, not just safety issues.
4. **Data residency.** European hub crew and passenger data had to remain processable under GDPR with data minimization; the Middle East hub had local data protection requirements restricting where PII could be persisted.
5. **Legacy integration.** The crew scheduling mainframe had no modern API; all integration had to go through an overnight batch file exchange plus a narrow, rate-limited SOAP interface the vendor charged per-call fees to expand.
6. **Budget.** Initial approved budget was $6.2M for year one, later reduced to $4.8M after a Q3 corporate cost-reduction mandate — this forced a significant scope cut, described in Section 7.
7. **No new PII exfiltration paths.** CISO Vasquez mandated that no crew or passenger PII could leave MGA's Azure tenant boundary, ruling out several vendor SaaS options during procurement.

---

5. Discovery Transcript

5.1 Kickoff Workshop — Week 1

Present: Priya Nadarajah (CTO), Tom Reyes (Chief Architect), Capt. Daniel Okoro (VP Flight Ops), Elena Vasquez (CISO), the Enterprise AI Architect (EAA), and two business analysts from Flight Ops.

Nadarajah: I want to be upfront about why we're doing this now and not two years ago. We had two IROPS events that cost this company real money and real reputational damage. The board doesn't want a research project. They want to know that eighteen months from now, when we get hit with a major winter storm, we don't fall apart the way we did in February of last year.

EAA: Understood. Before we talk architecture, I want to spend today and tomorrow just listening — to dispatch, to maintenance control, to crew scheduling. I've found that in aviation ops, the gap between what the org chart says the process is and what actually happens on a bad night is usually where the real requirements live.

Okoro: I'll say the quiet part out loud. I've sat through three vendor pitches in the last two years, all promising "AI-powered ops." None of them understood that a recommendation that looks 95% right and is wrong on the 5% — a legality violation, a fatigue call — isn't a minor bug. It's a felony exposure and a NTSB conversation. I need you to understand that before we go one more sentence into technology.

EAA: That's exactly the constraint I want to build the architecture around, not bolt on afterward. Can I ask — when your dispatchers make a recovery decision today, what's actually checked, and what's assumed?

Okoro: Crew legality is checked against Part 117 duty limits, but a lot of it is dispatcher memory plus a lookup tool that's frankly clunky. Aircraft swaps get checked against MEL — minimum equipment list — but cross-referencing which tail has which deferred maintenance item against which flight's ETOPS requirement is manual. On a bad night, we're doing this with spreadsheets and phone calls.

Vasquez: And I'll say my quiet part. I am not going to approve any architecture where a model has write access to crew scheduling or dispatch systems. Read access to build recommendations — fine, with controls. Autonomous write — no. That's not a negotiable starting position; it's where we start the conversation.

EAA: Good — that's useful to have explicit on day one. I'd actually push further: I don't think autonomous write access is warranted for a first release regardless of your position, because the ROI case doesn't require it. A dispatcher who gets a good recommendation in six minutes instead of doing manual work in forty-five is already transformative. Let's design for human-in-the-loop as the permanent state for anything crew- or MEL-adjacent, not a "phase one" we plan to remove later.

Reyes: That's the first time in eighteen months I've heard someone from the vendor side say that instead of trying to sell us autonomy.

5.2 Maintenance Control Shadow Session — Week 2

The EAA spent two full shifts in MGA's Maintenance Control Center, observing how AOG decisions were made.

Maintenance Controller (Sana Whitfield): Every morning I've got probably 200 open ACARS fault messages across the fleet. Most are noise — sensor glitches, transient faults that never repeat. Maybe three or four a week turn into an actual unscheduled removal. The problem is I can't tell which three or four until it's often too late, because the signal's buried.

EAA: Do you have access to the engine OEM's own predictive models — P&W's or CFM's diagnostics?

Whitfield: We get their alerts, sure, but they're tuned to protect the OEM, not us specifically — lots of false positives that trigger a borescope inspection "just in case," which costs us an aircraft on the ground for six hours for nothing. What I actually want is something that combines *our* dispatch reliability history, *our* specific tail number's maintenance record, and their sensor data into one signal I can trust.

This session directly shaped the decision (Section 9) to build a maintenance-prediction agent that fused OEM telemetry with MGA's own maintenance and dispatch-reliability history rather than relying solely on OEM alerting — a genuine differentiator from off-the-shelf predictive-maintenance SaaS tools MGA had evaluated and rejected in a prior RFP.

5.3 Business Capability Mapping — Week 3

The team ran a structured capability mapping session against MGA's operations value chain, identifying candidate AI opportunities and scoring them on a simple impact/feasibility matrix.

Capability	Current Maturity	AI Opportunity	Impact	Feasibility
Fleet health monitoring	Reactive, siloed	Predictive maintenance agent	High	Medium
Crew legality checking	Manual, rules-lookup	Legality-verification agent	High	High
Schedule recovery planning	Fully manual	Recovery orchestration agent	Very High	Medium
Passenger rebooking	Semi-automated (existing tool)	Out of scope — existing tool adequate	Low	N/A
Spare parts logistics	Manual forecasting	Deferred to phase 2	Medium	Low (data quality)
Crew communication/notification	Manual, phone-tree	Notification agent (phase 2)	Medium	High

The group explicitly deprioritized passenger rebooking automation (an existing tool was "good enough") and spare-parts forecasting (data quality in the parts inventory system was too poor to trust for a first release) — a discipline decision that kept scope from ballooning, revisited and validated again in Q3 when budget was cut.

5.4 ROI and Risk Discussion — Week 4

EAA: Let's put a number on this so we're not debating vibes with the board later. Unscheduled maintenance is costing $18M a year by your own finance team's estimate. If a prediction model gets you even a 15–20% reduction in unscheduled AOG events on the fleet segments with good sensor coverage — which is realistic based on what we've seen at comparable carriers — that's $2.7–3.6M in year one, growing as coverage expands.

CFO delegate (Marcus Webb, joining by video): And on the IROPS side?

EAA: Harder to quantify precisely because it's driven by rare, high-severity events, but your own numbers from the two storm events put single-event cost at $30–40M. If faster, more accurate recovery planning shaves even a few hours off the recovery timeline in a major event, or reduces the compensation-triggering legality misses, the ROI dwarfs the maintenance side — but it's lumpy and hard to forecast. I'd frame it to the board as: maintenance gives you a steady, provable annual return; IROPS gives you tail-risk reduction that pays for itself the first time it prevents a bad storm from becoming a catastrophic one.

Webb: The board will want the steady number more than the tail-risk story. Make sure the maintenance case can stand alone.

Risk identification from this session, captured formally in the risk register (Section 12): - Model risk: false-negative on maintenance prediction (missed real fault) is far more costly than false-positive - Regulatory risk: any crew-legality miscalculation is a compliance and safety exposure - Change management risk: dispatcher and maintenance controller workforce trust — both groups have seen "AI" pitched to them before by vendors who didn't understand the domain - Data risk: OEM telemetry contracts restrict how the data can be used/retained — legal review required - Vendor lock-in: engine OEM diagnostic APIs use proprietary formats

---

6. Architecture Workshops

6.1 Business & Information Architecture

The team modeled MGA's operations around two capability domains that would each get their own bounded agentic system, connected via an event backbone rather than a monolithic "ops AI":

- **Fleet Health Domain**: telemetry ingestion → fault correlation → predictive maintenance scoring → maintenance-control recommendation
- **Operations Recovery Domain**: disruption event detection → crew legality check → aircraft/crew/schedule recovery plan generation → dispatcher review

A shared **Operations Knowledge Graph** was proposed to sit underneath both domains, encoding entities (aircraft, tail numbers, crew members, certifications, MEL items, union rules, routes, airports) and their relationships, replacing what had been implicit knowledge in dispatchers' heads and scattered reference documents.

Information architecture debate — Week 6:

Enterprise Data Architect (Grace Lin): I want to push back on "shared knowledge graph." We already have a data warehouse. Why do we need a graph database on top of it?

EAA: Because the questions this system needs to answer are fundamentally relational and multi-hop: "which crew members are legal for this flight, given their duty history, their certifications for this aircraft type, their union's reserve rules, and the fact that the flight now needs to route through an airport their medical certificate doesn't currently cover" — that's five or six joins deep with rules that change based on context. You *can* do that in a relational warehouse with enough views and stored procedures, and honestly, for the legality-checking piece, a well-designed relational/rules-engine hybrid might be the right call. But for the recovery orchestrator reasoning about aircraft substitution chains and crew-pairing cascades, a graph representation is going to be both more maintainable and more explainable to a dispatcher asking "why did you recommend this."

Lin: So you're not proposing graph-for-everything.

EAA: No — and I'd actively push back on anyone who does. Crew legality rules are deterministic and auditable; that's a rules-engine problem, not an LLM-reasoning problem, and I don't want an LLM anywhere near computing whether someone has violated Part 117 duty limits. The graph is for the *search space* the recovery agent reasons over — aircraft substitution options, crew pairing alternatives — where the LLM's job is proposing candidate plans, and a deterministic rules engine validates every candidate before it's shown to a human.

This distinction — LLM for candidate generation, deterministic engine for legality validation — became the central architectural principle of the entire recovery domain and is referenced repeatedly in the ADRs (Section 17).

6.2 Data Architecture

Telemetry volumes were substantial: ACARS messages, engine OEM streaming sensor data (vibration, EGT margin, oil debris), and MGA's own maintenance work-order history. The team designed:

- **Ingestion**: Kafka topics per data source (ACARS, engine-OEM-A, engine-OEM-B, work-order-events), with schema registry enforcing contract discipline given two different OEM formats
- **Stream processing**: Flink jobs for windowed feature computation (rolling EGT margin trend, vibration anomaly scores) feeding a feature store
- **Batch layer**: nightly reconciliation against the maintenance work-order system (which only offered batch export) to backfill ground-truth labels for model retraining
- **Operations Knowledge Graph**: Neo4j, populated via CDC from the crew scheduling batch exports and a purpose-built sync service reading the legacy SOAP interface on a controlled polling schedule (rate-limited per the vendor contract — a real constraint the team had to design around rather than wish away)

6.3 AI/Platform Architecture — Whiteboard Session, Week 8

This session produced the reference architecture that governed the rest of the build. Reconstructed from the session notes:

Platform Architect (Sam Ito): Walk me through what happens end to end when a flight goes AOG at 2am.

EAA: [at whiteboard] Fault event lands on the ACARS Kafka topic. The Fleet Health stream processor has already been scoring this tail number continuously, so by the time the fault fires, we may already have an elevated risk score — that's the predictive half doing its job before the disruption even happens, ideally. Assume in this case it's a genuine surprise fault. The event triggers the Disruption Detection service, which publishes a `disruption.declared` event.

That event fans out to three things in parallel: the **Crew Legality Agent**, which pulls the affected flight's crew and starts computing who's still legal downstream; the **Aircraft Substitution Agent**, which queries the knowledge graph for spare aircraft in the right configuration at reachable airports; and the **Recovery Orchestrator**, which is the LangGraph-based agent that's actually going to assemble a coherent plan from what the other two produce.

Ito: Why three separate agents instead of one big agent with tools?

EAA: Two reasons. First, blast radius and testability — I want to be able to validate the crew legality logic in complete isolation, with deterministic test cases tied to actual regulatory citations, without that logic being entangled in a general-purpose reasoning loop. Second, and honestly the bigger reason: Capt. Okoro's team needs to trust each piece independently. If I hand Flight Ops a single opaque agent, they can't audit it. If I hand them "here is the legality-checking component, here's its test suite, here's the regulation citation for every rule," that's something their compliance team can actually review and sign off on section by section.

CISO Vasquez: And how do these agents talk to each other? I don't want to find out six months from now that there's some hidden channel bypassing my policy layer.

EAA: Agent-to-agent communication goes over an internal event bus using a structured task-request/task-result contract — not free-form natural language between agents. The Recovery Orchestrator issues a structured request to the Crew Legality Agent asking "is crew pairing X legal for reassignment to flight Y," and gets back a structured, typed response with a pass/fail and the specific regulatory basis if it fails. Every one of those calls is logged with a trace ID through our observability stack. Nothing agent-to-agent is hidden from you — it's actually more auditable than a single monolithic agent's internal chain-of-thought would be, because the contract is structured and the state is externalized at every hop.

Vasquez: I want that logging to be tamper-evident and retained for the same period as flight data recorder requirements — which is longer than your default retention policy probably assumes.

EAA: Noted — we'll size the observability data store retention to match, and we'll talk to Legal about whether these logs become discoverable records in the same way dispatch logs are, because I suspect they will, and we should design assuming that from day one rather than retrofit it.

6.4 Security & Identity Architecture

Given the CISO's hard line on write-access boundaries, the identity architecture was unusually central to this engagement.

- **Identity propagation**: every agent action was executed under a service identity, but any recommendation surfaced to a human carried the identity of the *requesting* dispatcher's session via OAuth 2.0 token exchange (RFC 8693), so downstream systems could attribute "who asked for this analysis" distinctly from "which system generated it"
- **OIDC** federation with MGA's existing Azure AD tenant for all human-facing surfaces (the dispatcher cockpit UI)
- **Authorization**: Open Policy Agent (OPA) with Rego policies enforcing a hard rule at the API gateway layer: no service identity associated with an agent could call a write-capable endpoint on the crew scheduling or dispatch system, full stop — enforced in infrastructure, not just application logic, specifically so it couldn't be quietly relaxed by a future engineer without a policy change that would show up in the OPA policy's own change history
- **Zero Trust network segmentation** between the AI platform VPC and the legacy mainframe integration layer, with the SOAP gateway as the only crossing point, itself behind mutual TLS and a dedicated service account with narrowly scoped, read-only credentials

Security Architect (Raj Mehta), Week 9 review:

Mehta: I ran a threat model on the agent-to-agent bus. My concern is prompt injection — if a fault message or

some upstream data field contains adversarial text, could that manipulate the Recovery Orchestrator's reasoning in a way that produces a plan that looks legitimate but embeds a bad decision?

EAA: Realistic concern, and worth walking through concretely. The data ACARS and OEM systems send is structured telemetry, not free text a human authored, so the injection surface there is narrow but not zero — sensor field names or diagnostic codes could theoretically be manipulated if someone compromised an upstream system, which is a different, bigger problem than prompt injection per se. The more realistic injection surface is actually free-text fields dispatchers themselves enter — notes, special instructions. We're treating all of that as untrusted input to the LLM layer: it's never concatenated directly into a system prompt with elevated trust, it's clearly delimited and the agent is instructed to treat it as reference context, not instructions. And critically — because every recommendation from the Recovery Orchestrator has to pass the deterministic Crew Legality Agent's validation before a human ever sees it, even a successfully manipulated recommendation can't produce an illegal crew assignment; it can at worst produce a bad *suggestion* that gets rejected by the rules engine or by the human reviewer.

Mehta: That defense-in-depth argument is the only reason I'm comfortable this doesn't need to go back to the drawing board. I want a specific test suite of adversarial inputs against the free-text fields before go-live, not just a design assertion.

This became ADR-007 and a mandatory gate in the AI Evaluation Strategy (Section 18).

6.5 Integration & API Strategy

- **API Gateway**: Kong, fronting all agent and service endpoints, enforcing the OPA policies above plus rate limiting on the OEM-facing calls (both to respect vendor contract terms and to control cost)
- **MCP adoption**: the team built an internal MCP server exposing the Operations Knowledge Graph and the legality rules engine as tools, consumed by the Recovery Orchestrator agent — chosen specifically so that the same tool interfaces could later be reused by a planned crew-notification agent (phase 2) without re-implementing integration logic, and so tool definitions, not ad hoc function-calling glue code, were the artifact under version control and review
- **A2A pattern**: structured, schema-validated task request/response contracts (not raw MCP tool calls) between the three agents described in 6.3, chosen deliberately over having one agent call the others' MCP tools directly, to preserve the clean audit boundary Vasquez required
- Legacy SOAP integration wrapped behind an internal REST/MCP adapter service, isolating the rest of the platform from the legacy interface's quirks and rate limits

---

7. Technical Debates

7.1 Build vs. Buy — the OEM Predictive Maintenance Offer

Both engine OEMs offered their own predictive-maintenance SaaS add-ons. This became a genuine, extended debate.

Reyes: P&W is offering us their EHM Plus package. It's proven, it's their own engine, presumably they know their own failure modes better than we ever could build ourselves. Why are we building custom?

EAA: Their model is tuned to protect their engine warranty exposure and their own fleet-wide statistics, not your specific dispatch-reliability economics. Whitfield told us this directly in discovery — their alerts generate inspections that are individually defensible but collectively too conservative for your on-time performance goals. If we buy their package as-is, we inherit their false-positive rate. What I'm proposing isn't "don't use their data" — it's "use their sensor data and their alert signal as one input feature among several, fused with your own history, so the final score is calibrated to your operation."

Reyes: That's more build than I think finance signed up for.

EAA: It's a real cost tradeoff, and I want to be honest about it rather than oversell a custom build. The counter-argument for buying as-is: faster time to value, lower engineering cost, OEM support. My recommendation is a middle path — license their data feed (not their whole decisioning package), and build the fusion/scoring layer ourselves, because that scoring layer is genuinely differentiated for you and it's not that large a build — a few months of ML engineering, not a platform.

This was escalated to the CFO and resolved as a hybrid: MGA licensed the raw telemetry feed from both OEMs (materially cheaper than the full decisioning package) and funded the in-house fusion model, which became one of the two components cut and re-scoped when the budget reduction hit in Q3 — reduced from "full fleet coverage" to "A350 and one A320 sub-fleet with best sensor coverage" for year one, expanding later.

7.2 LLM Provider Selection and Multi-Cloud Reality

MGA was already an Azure shop for corporate IT but ran significant workloads on AWS in the ops technology division for historical reasons (a prior acquisition).

Sam Ito (Platform Architect): Corporate mandate says Azure-first. But maintenance control's existing systems are AWS. Do we fight that mandate or work around it?

EAA: I'd avoid a religious war over cloud provider and instead ask what each domain actually needs. Azure AI Foundry gives you strong integration with your existing Azure AD identity model, which matters a lot given how central identity propagation is to Vasquez's requirements. But the Fleet Health domain's existing data pipeline is already AWS-native, and re-platforming that adds months and risk for no architectural benefit. My recommendation: Recovery domain agents run on Azure AI Foundry, consistent with corporate identity and the crew-scheduling integration's Azure footprint; Fleet Health's model training and inference stay on AWS where the data already lives, using Bedrock for the LLM-assisted diagnostic summarization piece. The two domains talk to each other over the same Kafka backbone regardless of cloud — that's a deliberate seam.

Ito: That's going to mean two LLM gateways, two sets of cost governance, two observability stacks.

EAA: It does, and I won't pretend that's free. What it buys you is not fighting a genuinely difficult, months-long re-platforming battle for a first release, and not creating a single point of failure across your entire ops stack. If in year two the operational overhead of running two clouds for this platform proves not worth it, that's a legitimate consolidation project — but I'd rather make that call with production experience than guess now.

This was accepted, with a shared LLM gateway abstraction layer (built in-house, thin wrapper) specifically so that model calls, cost tracking, and guardrail policy could be applied consistently even though the underlying cloud infrastructure differed — described further in ADR-004.

7.3 The Autonomy Debate, Revisited — Week 14

Midway through the build, with early prototype results looking strong, Reyes pushed to reconsider Vasquez's original hard line.

Reyes: The legality-checking agent's test suite is passing at 99.97% against our historical scenario library. At what point do we let it auto-approve the boring cases — a single reserve crew swap with no complications — and only escalate the genuinely ambiguous ones to a human?

Vasquez: 99.97% sounds great until you remember we run tens of thousands of these checks a year, and 0.03% is not zero incidents, it's a guaranteed handful of real ones. I have not moved off my position from week one.

Okoro: I'll actually take a position between you two, which might surprise people. I don't think blanket "always human in the loop forever" is right either, long-term — not because I trust the model more, but because I don't think an experienced dispatcher's attention should be spent rubber-stamping the genuinely trivial 80% of cases while the hard 20% get less scrutiny because reviewers are fatigued from clicking approve all day. That's a *human factors* argument for selective autonomy, not a "the model is good enough" argument.

EAA: That's actually the more interesting framing, and it changes what we should measure. If we ever revisit autonomy for the simplest case class, the gate shouldn't be "the model's accuracy crossed some number." It should be a joint safety-case submission, reviewed the same way you'd review a change to a paper-based dispatch procedure, with a defined rollback trigger, and — critically — instrumentation to detect reviewer fatigue and rubber-stamping in the *current* human-in-the-loop process, because if that's already happening, that's a live risk regardless of what we do about autonomy.

Vasquez: I'll agree to commission the fatigue study. I will not agree to any autonomy expansion in this release. Put it in the future roadmap, not the current scope.

This exchange is preserved in full because it shaped both the eventual incident response (Section 14) and the future roadmap (Section 20) — the fatigue-monitoring instrumentation Okoro requested was *not* built before go-live, a gap the postmortem later flagged directly (Section 15).

7.4 Evaluation Strategy Debate

ML Engineer (Devon Park): For the recovery plan generation, how do we even define "correct"? There's rarely one right answer to a recovery scenario.

EAA: Agreed, and I'd resist the temptation to build a single accuracy metric here. We need three different evaluation lenses. First, *hard constraint satisfaction* — does the plan violate any legality or MEL rule — which is binary and must be 100%, enforced by the deterministic layer, not the eval suite catching failures after the fact. Second, *plan quality against historical dispatcher decisions* — using a curated set of past real disruptions with known outcomes, scored by actual dispatchers blind to whether a given plan came from the agent or a human colleague, to avoid anchoring bias. Third, *operational efficiency metrics* in shadow deployment — time-to-plan, number of downstream misconnects the plan produces if hypothically executed against historical data.

Park: Blind human evaluation is expensive and slow.

EAA: It is, and we're not running it continuously — we run it as a release gate, not a CI check. Day-to-day regression testing uses the constraint-satisfaction suite and a smaller automated plan-quality heuristic. The blind dispatcher evaluation happens before each material model or prompt change ships to production, not on every commit.

---

8. Executive Reviews

8.1 Architecture Review Board — Week 11

The ARB session was the most contentious governance gate in the engagement.

Chief Architect Reyes (chairing): We're here to approve or reject the proposed multi-agent architecture for production build-out. Elena, you have concerns on record — do you want to restate them for the board?

Vasquez: My core concern isn't technical competence, it's blast radius. I want a formal answer to: what is the worst thing this system can cause to happen, end to end, and what stops it?

EAA: Fair, and I'll answer directly rather than defensively. The worst case for the Recovery domain is a plan reaching a human dispatcher that looks correct but contains a legality violation they don't catch because they're fatigued or the UI presents it in a way that obscures the issue. The controls: the deterministic legality engine is the sole source of truth for pass/fail, not the LLM's own claim about legality — the UI is required to surface the rules-engine verdict prominently, not bury it in agent-generated prose. Second control: no write access from any agent to the systems of record, meaning even a bad human approval still goes through the same downstream systems and checks that exist today. Third: full trace logging of every agent decision and every human override, reviewed on a rolling audit basis, not just when something goes wrong.

Vasquez: What's the worst case for Fleet Health?

EAA: A false negative — missing a real developing fault — which is a defense-in-depth argument I can't make as strong, honestly. Mitigation there is that this system augments, it doesn't replace, existing scheduled maintenance intervals and OEM alerting; it's a prioritization and early-warning layer on top of a maintenance program that continues operating on its existing safety basis regardless. We are not proposing to relax any existing maintenance interval based on this system's output in year one.

Okoro: I'll go on record supporting this architecture, conditional on the legality-engine test suite being reviewed by our own compliance team line by line before go-live, not just accepted based on your team's testing.

Vasquez: Conditional approval from me too, same condition, plus I want the tamper-evident logging requirement written into the SOW as a contractual deliverable, not a nice-to-have.

Reyes: Approved with those two conditions. Let's move to build.

8.2 CIO Budget Review — Week 22 (Mid-Engagement)

This session is where the Q3 budget reduction hit.

CIO (Marcus Webb, now formally CIO after a reorg): I need to cut $1.4M from this program's remaining budget. Corporate is asking every major initiative to find 20%. Walk me through what happens to scope.

EAA: I'd rather cut scope deliberately than cut quality across the board. Here's the tradeoff as I see it: full fleet coverage for predictive maintenance in year one requires onboarding telemetry from all three fleet types, including the legacy 737NG fleet that's being retired within three years anyway. I'd propose we drop 737NG from year-one scope entirely — it has the worst sensor coverage of the three fleets and the shortest remaining service life, so the ROI case for building predictive models against it was always weakest. That alone is close to $400K in reduced integration work.

Webb: What else?

EAA: The originally scoped crew-notification agent — automatically drafting and sending disruption notifications to affected crew — was phase-two scope already, but some of its groundwork was pulled into year one for efficiency. I'd push that fully back to phase two; it's a nice-to-have on top of the recovery orchestrator, not core to it. That's another meaningful chunk. Between those two, plus a leaner initial observability tooling footprint that we can expand post-launch rather than building all its bells and whistles up front, I believe we get close to your target without touching the core legality-engine or recovery-orchestrator scope, which is where the real safety and ROI value is concentrated.

Webb: I want it in writing that safety-relevant scope wasn't what got cut.

EAA: Agreed — and I'd actually recommend that framing go back to the ARB as a formal scope-change

notification, not just a budget memo, given Elena’s conditions from the review.

This budget reduction and the resulting scope cuts are documented as ADR-011.

8.3 CISO Sign-off Review — Week 30

Prior to go-live, Vasquez required a final security review.

Mehta: Penetration test against the API gateway and OPA policy layer is complete. One finding: a misconfigured policy briefly allowed the Aircraft Substitution Agent’s service identity broader read scope on the crew scheduling read-replica than intended — not a write vulnerability, but broader read than the least-privilege design called for. Fixed and retested.

Vasquez: Adversarial prompt-injection test suite against the free-text fields, as required from the Week 9 review?

Devon Park: 340 adversarial test cases run against the Recovery Orchestrator. Zero produced a plan that passed the deterministic legality check while containing an actual violation — which is the property that matters. Eleven produced degraded-quality plans (nonsensical or clearly wrong aircraft substitutions) that were correctly rejected downstream, either by the rules engine or flagged by the confidence-scoring layer for mandatory human review.

Vasquez: That’s the number I needed to see. I’m signing off, with the standing requirement that this adversarial suite reruns on every model or prompt version change, not just at initial go-live — I want that written into the operational runbook, not left as tribal knowledge.

---

9. Final Architecture

The production architecture, as it shipped:

Fleet Health Domain - Kafka-based ingestion of ACARS, dual-OEM engine telemetry, and maintenance work-order events - Flink streaming feature computation feeding a feature store (Feast) - Fusion prediction model (gradient-boosted ensemble, not an LLM) producing calibrated risk scores per tail number/component, running on AWS - An LLM-based diagnostic summarization agent (Bedrock) that translates model output plus recent fault history into a plain-language briefing for maintenance controllers — explicitly NOT the decision-making component, purely an explanation/communication layer - Human maintenance controller retains all dispatch decision authority

Operations Recovery Domain - Disruption Detection service (event-driven, Kafka) publishing `disruption.declared` events - Crew Legality Agent: deterministic rules engine (not LLM-based) encoding Part 117, EASA FTL, and both union contracts, exposed as an MCP tool; an LLM component sits *in front* of it only to translate structured dispatcher queries into rules-engine calls and to render results in natural language — never to compute legality itself - Aircraft Substitution Agent: queries the Operations Knowledge Graph (Neo4j) for viable spare aircraft, filtered by MEL status, configuration, and reachability - Recovery Orchestrator: LangGraph-based multi-step agent on Azure AI Foundry, calling both other agents via structured A2A task contracts, assembling candidate recovery plans, and calling out to the legality engine to validate every candidate before ranking - Dispatcher Cockpit UI: presents ranked candidate plans with the legality-engine verdict prominently displayed (per the ARB requirement), full reasoning trace available on demand, mandatory human approval before any downstream system action

Cross-cutting - OPA/Rego policy enforcement at the Kong API gateway; hard block on write access from any agent identity to systems of record - OAuth 2.0 / OIDC identity federation with token exchange for attributing human accountability to every action - Tamper-evident, long-retention observability store (OpenTelemetry traces, retained per flight-data-recorder-equivalent policy) - Shared LLM gateway abstraction for cost tracking and guardrail consistency across the Azure/AWS split - Semantic caching on the diagnostic-summarization agent (high repeat-query rate during active disruption events) to control both cost and latency during the exact moments when speed matters most

---

10. Delivery Roadmap

Phase	Duration	Scope
Discovery & Architecture	Months 1–3	Discovery, ARB approval, foundational data pipelines
Fleet Health MVP	Months 3–6	A350 + one A320 sub-fleet, fusion model, shadow deployment against historical data
Fleet Health GA	Months 6–8	Production rollout, maintenance controller training, feedback loop instrumentation
Recovery Domain Build	Months 5–9	Legality engine, knowledge graph, orchestrator build (overlapped with Fleet Health GA)
Recovery Shadow Deployment	Months 9–10	Running in parallel with live dispatcher decisions, no production influence, comparing agent recommendations against actual dispatcher outcomes
Recovery GA — Hub 1	Month 10	Single hub go-live, mandatory human approval workflow
Incident & Remediation	Month 14 (post-GA)	Production incident (Section 14), HITL redesign
Recovery GA — Hub 2 expansion	Month 16	Expansion following remediation, revised approval workflow

11. Risks

Final risk register at go-live (abbreviated to material items):

Risk	Likelihood	Impact	Mitigation	Owner
Legality-engine rule drift (union contract renegotiation not reflected)	Medium	High	Quarterly rules-engine reconciliation review with Labor Relations	Compliance Officer
Dispatcher over-trust / automation complacency	Medium-High	High	UI design surfacing confidence and rules-engine verdict explicitly; planned fatigue-monitoring instrumentation (not built for GA — see Lessons Learned)	Flight Ops
OEM telemetry contract renewal risk / vendor lock-in	Low	Medium	Fusion model architected so OEM feed is a swappable input feature, not a hard dependency	Enterprise Architect
Cross-cloud (Azure/AWS) operational complexity	Medium	Medium	Shared gateway abstraction; single on-call runbook covering both	Platform Architect
Data residency violation (EU/ME crew PII)	Low	Very High	Regional data segregation enforced at ingestion; legal review of every new data flow	CISO
Prompt injection via free-text fields	Low (post-mitigation)	High	Adversarial test suite, rerun on every model change; deterministic validation as backstop	Security Architect

12. Governance Model

The governance model formalized after the ARB debates:

- **AI Change Advisory Board**: any change to the Crew Legality rules engine, the Recovery Orchestrator's prompts/models, or the confidence-scoring thresholds requires sign-off from Compliance, Flight Ops, and the AI platform lead — modeled explicitly on MGA's existing procedure change-control process, not invented fresh, so it would feel familiar to the operational stakeholders who had to use it
- **Standing HITL requirement**: codified as a permanent architectural principle, not a phase-one placeholder — any output touching crew duty-time legality or MEL dispatch requires human sign-off, full stop, with no roadmap item to remove it (the "selective autonomy" idea from Section 7.3 remains parked pending the fatigue study, not approved)
- **Audit cadence**: monthly sampling review of agent recommendations vs. human decisions, with any override pattern (humans consistently rejecting a certain plan type) triggering a model/prompt investigation
- **Regulatory correspondence protocol**: Legal designated as the point of contact for any question of whether system outputs constitute discoverable records; observability retention set accordingly

---

13. Production Rollout

Hub 1 go-live (Month 10) was executed as a phased cutover:

- Week 1–2: shadow mode continued in production (real data, no dispatcher-facing output) as a final validation
- Week 3: opt-in rollout to two senior dispatchers per shift, with the EAA team on-site
- Week 5: full shift rollout at Hub 1, mandatory training completed by all dispatchers (a two-day program built jointly with Flight Ops, focusing heavily on “how to read and challenge a recommendation,” not just “how to click approve”)
- Week 8: Fleet Health maintenance-controller rollout completed at both original hubs

Early production metrics tracked closely against the shadow-deployment baseline to catch any regression between simulated and live performance — a discipline the team insisted on specifically because several past MGA vendor tools had looked good in demo and degraded once exposed to live data quirks.

---

14. Production Incident — Month 14

14.1 Incident Summary

During a significant winter weather event at Hub 1, the Recovery Orchestrator was handling a high volume of simultaneous disruptions — 31 affected flights within a four-hour window, near the upper end of what shadow-mode testing had validated but not beyond it. A dispatcher, working her ninth consecutive hour of an extended shift authorized under IROPS emergency staffing provisions, approved a crew reassignment plan generated by the orchestrator.

The plan reassigned a first officer to an additional flight leg that, combined with the officer's earlier duty period *that had itself been extended by an unrelated delay after the plan was generated but before it was executed*, would have exceeded Part 117 duty-time limits by 22 minutes at scheduled arrival. The legality engine had correctly validated the plan *at the moment it was generated*, based on the schedule as it stood then — the violation only materialized because the officer's prior flight leg was further delayed after the recommendation was approved, and the system had no mechanism to re-validate a previously-approved plan against subsequent schedule changes.

The first officer's own personal duty-time tracking (a legal requirement pilots maintain themselves) caught the discrepancy before departure. The flight was held, a replacement crew member was called in, and the flight departed 90 minutes late. No regulatory violation occurred — the flight never operated outside legal limits — but MGA determined this was a reportable near-miss under its internal safety management system (SMS) and voluntarily disclosed it to the FAA under the Aviation Safety Action Program (ASAP), which protects good-faith voluntary disclosures.

14.2 Incident Response Transcript

Vasquez (emergency session, convened within 6 hours): I want the timeline, not speculation. What did the system actually do wrong?

Devon Park: Technically, nothing wrong at the moment of decision — the legality engine validated the plan correctly against the data it had. The gap is architectural: we validate a plan once, at generation and approval time, and then treat it as static. We never designed for the case where the world changes *between* approval and execution in a way that invalidates an already-approved plan.

Okoro: That's not a technical gap to me, that's a process gap we should have caught in the safety case. In manual operations, a dispatcher re-checks legality if anything changes before departure, as a matter of training and habit. We built a system that made the "check once" behavior look authoritative and final in a way that maybe made the human less likely to re-verify, not more.

EAA: I think that's the right diagnosis, and I want to own that we didn't design for revalidation-on-change, and we should have. I'd add one more contributing factor worth being honest about: the dispatcher was nine hours into an extended shift. That's exactly the fatigue-and-automation-complacency risk Capt. Okoro raised in Week 14 of the build, and the fatigue-monitoring instrumentation we discussed then and agreed to defer was never built before go-live. I don't think it's fair to call this "the model's fault" or "the dispatcher's fault" — it's a systemic gap between a known risk we identified early and a mitigation we deprioritized under budget and schedule pressure.

Vasquez: I appreciate you not being defensive about that. I want to know: is this a "pause the system" moment?

Okoro: My honest view — no, not a full pause, because the underlying legality check performed correctly given its inputs, and a full pause during active winter ops creates its own safety risk by forcing an overloaded manual process back onto exhausted dispatchers. But I want an immediate compensating control today, not next sprint.

EAA: Agreed. Immediate mitigation I'd propose tonight: any approved-but-not-yet-executed plan gets automatically re-validated against the legality engine if *any* upstream schedule change touches a crew member in that plan, with a hard block on execution and mandatory re-approval if the re-check fails. That's containable to build and test in days, not months, because we're reusing the existing legality engine — we're just changing when it's invoked, adding an event trigger on schedule-change events for crew already in an approved plan.

Vasquez: Do that as an emergency change, expedited through the AI Change Advisory Board tonight, not the normal cadence. And I want a full postmortem with root cause, not just the compensating control.

14.3 Compensating Control Deployment

The re-validation trigger was designed, tested against the adversarial and regression suites, reviewed by an expedited Change Advisory Board session, and deployed within 96 hours — itself a case study in the value of the deterministic-engine-as-backstop architecture: the fix didn’t require retraining or reprompting the LLM orchestrator at all, only adding a new event trigger for the already-existing, already-trusted legality engine.

15. Lessons Learned

Captured verbatim from the formal postmortem, held two weeks post-incident with the full ARB:

1. **“Validate once, trust forever” is a latent design flaw in any safety-adjacent recommendation system operating in a dynamic environment.** The architecture correctly separated LLM reasoning from deterministic validation, but implicitly assumed validation state remained valid until execution — an assumption that held in testing (where scenarios were static) and failed in production (where the world kept changing). Any future safety-adjacent system in this domain must treat validation as continuously re-checkable against triggering events, not a one-time gate.

2. **Deferred risk items need an expiration date and an owner, not just a parking lot.** The fatigue-monitoring instrumentation was explicitly identified as a risk mitigation in Week 14 of the build and explicitly deferred under budget pressure in Week 22 — but nothing forced a re-review of that deferral before go-live. The governance model now requires every deferred risk-mitigation item to be re-reviewed by the ARB before any GA milestone, not just logged and forgotten.

3. **Shadow-mode testing validated volume and legality-check correctness but not temporal dynamics.** The team’s evaluation strategy (Section 18) rigorously tested “is this plan correct given this snapshot of the world” but never tested “does this plan remain correct as the world continues to change after generation.” This is now a mandatory evaluation category for any future recovery-domain change.

4. **The defense-in-depth architecture worked as designed, even though the incident happened.** It’s worth stating plainly in the postmortem: the reason this was a 90-minute delay and a voluntary disclosure rather than an actual regulatory violation is that a human pilot’s own independent duty-time tracking — a redundant, non-AI safety layer — caught what the AI-assisted process missed. This validates the principle of never relying on a single layer of protection, AI or otherwise, and argues against ever removing independent human verification steps in the name of efficiency.

5. **Cross-functional trust, built over eleven months of hard conversations, is what made the incident response fast and honest rather than defensive and slow.** Vasquez’s Week 1 hard line on write access, and the entire team’s discipline in respecting it even under schedule pressure, is why this incident was containable within hours rather than becoming an actual regulatory event. Okoro noted in the postmortem that this was the first vendor engagement in his tenure where the technical team volunteered the fatigue-risk connection before being asked.

---

16. Enterprise Architecture Artifacts

Representative artifacts produced and maintained as living documents (full versions retained in MGA's architecture repository; summarized here):

- **Capability Map**: Fleet Health / Operations Recovery capability domains, mapped against the full MGA operations value chain (referenced in Section 5.3)
- **Context Diagram**: system boundary showing the AI platform's interfaces to crew scheduling mainframe, ACARS network, dual-OEM telemetry feeds, dispatcher cockpit, and maintenance-control workstations
- **Domain Event Catalog**: canonical schema registry entries for `disruption.declared`, `plan.generated`, `plan.approved`, `legality.revalidation.triggered`, and fourteen other domain events, each with owning team and schema version history
- **Identity & Access Model**: full RBAC/ABAC matrix cross-referencing agent service identities, human roles, and OPA policy bindings
- **Data Flow Diagram with Residency Annotations**: every PII-bearing data flow annotated with its residency constraint and legal basis, maintained jointly with Legal

---

17. Architecture Decision Records (ADRs)

ADR-001: Separate deterministic rules engine from LLM-based reasoning for all legality determinations. Status: Accepted. Context: LLM reasoning is not verifiably reliable enough for binary regulatory compliance determinations. Decision: all Part 117/EASA FTL/union-contract legality checks run through a deterministic, unit-testable rules engine; LLM components may query and explain results but never compute them. Consequences: higher upfront engineering cost to build and maintain the rules engine as regulations and contracts change; substantially higher auditability and the architectural backstop that contained the Month 14 incident.

ADR-002: Multi-agent decomposition (legality / substitution / orchestration) over a single monolithic agent. Status: Accepted. Consequences: higher integration complexity and A2A contract maintenance burden; independently testable and auditable components, which proved decisive for both the ARB approval and MGA compliance team's willingness to sign off.

ADR-003: No write access from any agent identity to systems of record. Status: Accepted, non-negotiable per CISO mandate. Consequences: all outputs are recommendations requiring human execution through existing systems; slower per-transaction throughput than a fully autonomous system would offer, judged an acceptable and appropriate tradeoff given the domain.

ADR-004: Split cloud footprint (Azure for Recovery domain, AWS for Fleet Health domain) with a shared LLM gateway abstraction. Status: Accepted. Alternatives considered: full re-platform to single cloud (rejected — cost/schedule); fully independent stacks with no shared abstraction (rejected — would fragment cost governance and guardrail policy). Consequences: operational overhead of dual-cloud on-call, offset by avoiding a high-risk re-platforming project.

ADR-005: Knowledge graph (Neo4j) for aircraft/crew substitution search space; relational/rules-engine hybrid for legality determination. Status: Accepted. See Section 6.1 debate.

ADR-006: License raw OEM telemetry feeds rather than full OEM predictive-maintenance decisioning packages; build fusion model in-house. Status: Accepted. See Section 7.1.

ADR-007: Mandatory adversarial prompt-injection test suite for all free-text input surfaces, rerun on every model/prompt version change. Status: Accepted, contractually mandated per CISO sign-off (Section 8.3).

ADR-008: Structured, schema-validated A2A task contracts between agents; no free-form natural-language inter-agent communication. Status: Accepted. Consequences: some loss of flexibility in agent-to-agent reasoning compared to free-form approaches; substantial gain in auditability, directly requested by CISO.

ADR-009: Semantic caching on the diagnostic-summarization and disruption-briefing agents. Status: Accepted. Rationale: repeat-query patterns during active disruption events (multiple dispatchers/controllers querying similar situations) made caching a meaningful cost and latency win precisely during the highest-value, highest-time-pressure moments.

ADR-010: Standing, permanent human-in-the-loop requirement for all crew-legality and MEL-dispatch-adjacent outputs; no roadmap path to remove it. Status: Accepted, revised and hardened post-incident (originally framed as a "phase one" position in Week 1, formally made permanent in the post-incident governance update, Section 12).

ADR-011: Q3 budget-reduction scope cuts — drop 737NG fleet from year-one predictive-maintenance scope; defer crew-notification agent to phase two. Status: Accepted. See Section 8.2.

ADR-012 (post-incident): Approved recovery plans must be automatically re-validated against the legality engine if any upstream schedule change affects a crew member in that plan, with hard execution block pending re-approval on re-check failure. Status: Accepted, emergency-deployed. See Section 14.3.

---

18. AI Evaluation Strategy

- **Constraint satisfaction (mandatory, 100% target, gates every release)**: automated regression suite of historical and synthetic disruption scenarios verifying zero legality-rule violations pass through undetected
- **Adversarial robustness (mandatory, gates every release)**: 340+ case adversarial prompt-injection suite against all free-text input surfaces (Section 8.3), expanded post-incident to include temporal-drift scenarios (Section 15, lesson 3)
- **Plan quality — blind dispatcher evaluation (release gate, not continuous CI)**: curated historical disruption scenarios scored blind by practicing dispatchers, comparing agent-generated and human-generated plans without revealing source
- **Operational efficiency — shadow deployment metrics**: time-to-plan, simulated downstream misconnect count, compared against live baseline before any GA expansion
- **Temporal-dynamics evaluation (added post-incident)**: scenario suite specifically testing plan validity as the modeled world state changes after plan generation/approval — the category the postmortem identified as missing
- **Ongoing production monitoring**: monthly audit sampling of human override patterns (Section 12); confidence-score calibration drift tracking on the Fleet Health fusion model against realized outcomes

---

19. Operational Runbook

Representative runbook entries (abbreviated):

- **On-call escalation**: joint Azure/AWS on-call rotation with a single unified alerting channel; escalation path distinguishes “platform incident” (infrastructure) from “AI quality incident” (a specific runbook branch requiring both the AI platform lead and Flight Ops/Compliance for anything touching legality or MEL outputs)
- **Model/prompt change deployment procedure**: mandatory AI Change Advisory Board sign-off; mandatory rerun of full adversarial and constraint-satisfaction suites; staged rollout starting with shadow mode before dispatcher-facing exposure
- **Re-validation trigger monitoring**: dashboard tracking frequency and resolution of the post-incident re-validation mechanism (ADR-012), reviewed weekly by Flight Ops during the first quarter post-deployment, then folded into the standard monthly audit cadence
- **OEM data feed outage procedure**: fallback behavior when either engine OEM’s telemetry feed is degraded or unavailable — fusion model degrades gracefully to MGA-internal-data-only scoring with a visible confidence downgrade flag, rather than failing silently
- **Emergency pause procedure**: documented conditions and authority (jointly held by CISO and VP Flight Ops) for pausing Recovery Orchestrator dispatcher-facing output and falling back to fully manual process, with defined communication protocol to affected shifts

---

20. Future Roadmap

Items explicitly deferred, with the reasoning preserved so future teams understand *why*, not just *what*:

1. **Fatigue and automation-complacency monitoring instrumentation.** Originally raised Week 14, deferred under budget pressure Week 22, directly implicated in the Month 14 incident. Now a committed Q1 next-fiscal-year priority with explicit ARB ownership, not merely a backlog item.
2. **Selective autonomy for the lowest-complexity legality-check case class** (Section 7.3). Remains parked pending the fatigue study above — Vasquez's position (no autonomy expansion without it) stands, and the post-incident environment makes near-term reconsideration unlikely; the roadmap explicitly frames this as multi-year, not next-release.
3. **Crew-notification agent** (deferred from year one, Section 8.2), to reuse the existing MCP tool interfaces built for the Recovery domain.
4. **737NG fleet predictive-maintenance coverage**, likely to be judged not worth building given the fleet's retirement timeline — a standing recommendation to formally de-scope rather than perpetually defer.
5. **Hub 3 and Hub 4 expansion** (Middle East and secondary North American hub), contingent on data residency architecture review specific to the Middle East hub's local requirements, not yet completed.
6. **Spare-parts logistics forecasting**, deprioritized in original discovery (Section 5.3) due to poor source data quality; a parallel data-quality remediation program in the parts inventory system is a prerequisite, not yet funded.
7. **Cross-airline benchmarking within the holding company.** MGA's parent holding company owns two other regional carriers observing this engagement's outcome; a decision on whether to replicate the architecture (versus each carrier's materially different union contracts and fleet mix warranting a fresh discovery phase) is pending as of this writing.